Coursework Portfolio 1 — Week 7 Lab Report

Web Application Vulnerabilities: Juice Box

Student ID: SHE23601567

Module Code: CMP-X305-0

Submission Deadline: 31 March 2026

University of Roehampton London

Contents

Web Application Vulnerabilities: Juice Box 1
Introduction 6
Part 1: Lab Environment Setup 7
Step 1: OWASP Juice Shop VM - Installed juice shop locally to the lab computers..... 7
Part 2: Complete Challenges 9
2.1 Score Board (Miscellaneous) 9
2.2 DOM XSS (Cross Site Scripting) 11
2.3 Reflected XSS (Cross Site Scripting) 12
2.4 Login MC SafeSearch 14
2.5 Privacy Policy (Miscellaneous) 16
2.6 Login Admin (SQL Injection) 17
2.7 Admin Section (Security Misconfiguration) 19
2.8 Login Jim (SQL Injection) 20
2.9 Bjoern's Favorite Pet (Broken Authentication) 21
2.10 Kill Chatbot (Vulnerable Components) 25
Conclusion 28
Reflection 29
Use of Artificial Intelligence 30

Table of Figures

Figure 1: Cloning repository and navigating to juice-shop folder 7
Figure 2: Command "npm install" to Install necessary dependencies 7
Figure 3; Juice Shop application successfully launched on port 3000 8
Figure 4: OWASP Juice Shop web application accessible via browser at localhost:3000..... 8
Figure 5: Browser developer tools open on the Juice Shop application 9
Figure 6: Searching for "score" within main.js reveals the hidden "/score-board" route embedded in the client-side source code, exposing an unprotected administrative path. 10
Figure 7: Score-board showing score board challenge completed 10
Figure 8: XSS alert triggered successfully via the search bar 11
Figure 9: DOM XSS challenge successfully completed using an iframe injection payload... 11
Figure 10: Search functionality returning results for "apple", showing the query reflected in the URL parameter. 12
Figure 11: XSS payload injected via URL parameter, triggering an alert and confirming the search input is unsanitised 12
Figure 12: DOM XSS payload executing via the track-result URL 13
Figure 13: Reflected XSS challenge successfully completed, scoreboard showing 3 challenges solved. 13
Figure 14: Google search for "MC SafeSearch" used to gather publicly available OSINT on the target user. 14
Figure 15: Video lyrics revealing the dog's name "Mr. Noodles" — used to derive the password "Mr. N00dles" 14
Figure 16: Logging in as MC SafeSearch using the discovered password "Mr. N00dles". ... 15
Figure 17: Login successful — browser warning confirms the password was found in a known data breach 15
Figure 18: Account menu showing Privacy & Security section containing the Privacy Policy link. 16
Figure 19: Privacy Policy page accessed at the privacy-security route, triggering the challenge completion 16
Figure 20: Successfully solced Privacy Policy challenge 16
Figure 21: Login page of the OWASP Juice Shop 17
Figure 22: Admin account access granted confirming successful SQL injection login bypass. 17
Figure 23: Successfully solved Login Admin challenge 17
Figure 24: SQL injection payload ' OR 1=1-- entered in the email field to bypass authentication 17
Figure 25: Administration panel accessed directly via /administration route without authorisation. 19
Figure 26: Admin Section challenge completed, exposing all registered users and customer feedback. 19
Figure 27: SQL injection payload entered targeting Jim's account using his known email to bypass authentication. 20

Figure 28: Login Jim challenge successfully completed via SQL injection. 20
Figure 29: Login attempt for bjoern@owasp.org confirming the account exists 21
Figure 30: Forgot Password page revealing the security question "Name of your favorite pet?" for bjoern@owasp.org. 21
Figure 31: Photo Wall revealing the Twitter handle @bkimminich linked to Bjoern's account. 22
Figure 32: Björn Kimminich's public Twitter profile confirmed via OSINT. 22
Figure 33: Tweet revealing Bjoern's pet name "Zaya" 23
Figure 34: Password reset form completed using "Zaya" as the security question answer. . 23
Figure 35: Successfully solved Bjoern's favorite pet challenge 24
Figure 36: Juice Shop's public GitHub repository revealing the chatbot source code for analysis. 25
Figure 37: Search for "vm" revealing the vulnerable VM instance in the chatbot code. 26
Figure 38: addUser method passing unsanitised user input directly into the VM 26
Figure 39: Navigation menu showing the Support Chat option 27
Figure 40:VM injection payload sent via chatbot, crashing the process and confirming the vulnerability. 27
Figure 41: Successfully solved kill chatbot challenge 27

Introduction

The aim of this lab was to identify, exploit, and understand common web application vulnerabilities using the OWASP Juice Shop in order to analyse security weaknesses and their impact on web application security.

Part 1: Lab Environment Setup

Step 1: OWASP Juice Shop VM - Installed juice shop locally to the lab computers.

For this step, I cloned the OWASP Juice Shop repository from GitHub to my local machine using the command `git clone https://github.com/juice-shop/juice-shop.git --depth 1`. After navigating to the cloned folder with `cd juice-shop`.

```
PS C:\Users\student> git clone https://github.com/juice-shop/juice-shop.git --depth 1
Cloning into 'juice-shop'...
remote: Enumerating objects: 1272, done.
remote: Counting objects: 100% (1272/1272), done.
remote: Compressing objects: 100% (869/869), done.
remote: Total 1272 (delta 380), reused 1260 (delta 380), pack-reused 0 (from 0)
ects:  99% (1260/1272), 25.36 MiB | 50.69 MiB/s
Receiving objects: 100% (1272/1272), 46.35 MiB | 51.71 MiB/s, done.
Resolving deltas: 100% (380/380), done.
PS C:\Users\student> cd juice-shop
```

Figure 1: Cloning repository and navigating to juice-shop folder

I installed all necessary dependencies with the command `npm install`. Once the installation was complete, I started the application using `npm start`, which launched the OWASP Juice Shop on my local server. I then accessed the application by opening a browser and navigating to `http://localhost:3000`. The screenshot shows the terminal during this process, confirming the successful setup and execution of the application.

```
PS C:\Users\student\juice-shop> npm install
npm warn deprecated @npmcli/move-file@1.1.2: This functionality has b
npm warn deprecated inflight@1.0.6: This module is not supported, and
lru-cache if you want a good and tested way to coalesce async request
prehensive and powerful.
npm warn deprecated eivindfjeldstad-dot@0.0.1: Use @eivifj/dot instea
npm warn deprecated formatio@1.1.1: This package is unmaintained. Use
npm warn deprecated samsam@1.1.2: This package has been deprecated in
npm warn deprecated source-map-url@0.4.1: See https://github.com/lyde
npm warn deprecated urix@0.1.0: Please see https://github.com/lydell/
npm warn deprecated resolve-url@0.2.1: https://github.com/lydell/reso
```

Figure 2: Command "npm install" to Install necessary dependencies

```
> juice-shop@19.1.1 start
> node build/app

info: Detected Node.js version v22.22.0 (OK)
info: Detected OS win32 (OK)
info: Detected CPU x64 (OK)
info: Configuration default validated (OK)
info: Entity models 20 of 20 are initialized (OK)
info: Required file server.js is present (OK)
info: Required file index.html is present (OK)
info: Required file main.js is present (OK)
info: Required file runtime.js is present (OK)
info: Required file tutorial.js is present (OK)
info: Required file vendor.js is present (OK)
info: Required file styles.css is present (OK)
info: Port 3000 is available (OK)
info: Chatbot training data botDefaultTrainingData.json validated (OK)
info: Domain https://www.alchemy.com/ is reachable (OK)
info: Server listening on port 3000
```

Figure 3; Juice Shop application successfully launched on port 3000

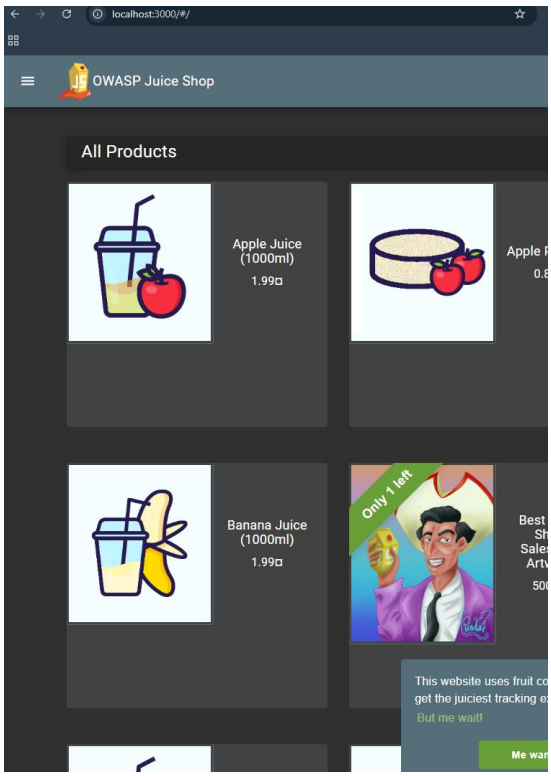

Figure 4: OWASP Juice Shop web application accessible via browser at localhost:3000

Part 2: Complete Challenges

1. The name of the challenge.
2. The difficulty star rating of the challenge.
3. The evidence in the form of a screenshot

2.1 Score Board (Miscellaneous)

- Difficulty star: 1

1. Browser Developer Tools were opened using the F12 key to inspect the client-side code of the application.The **Sources** tab was used to view the application's JavaScript files. The file main.js was located, which contains the compiled frontend code.

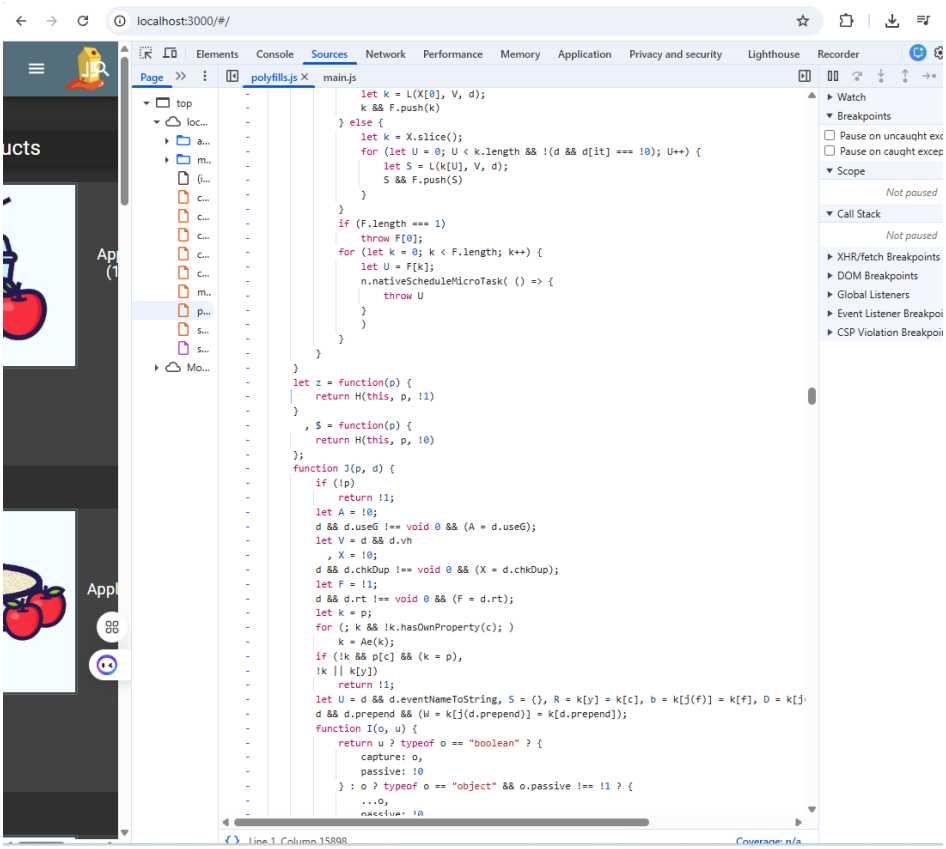

Figure 5: Browser developer tools open on the Juice Shop application

2. The search function (Ctrl + F) was used to locate the keyword **score**, which revealed the hidden route `score-board` within the application code.

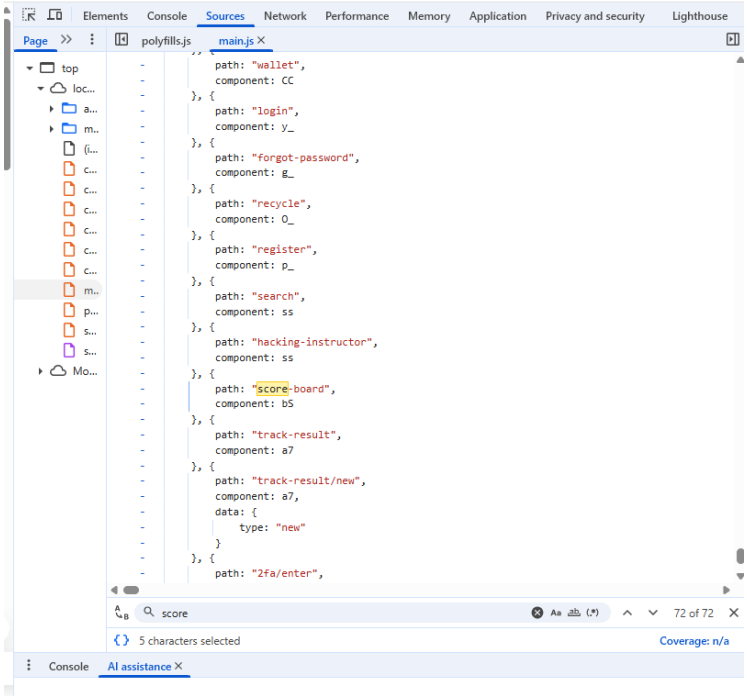

Figure 6: Searching for "score" within main.js reveals the hidden "/score-board" route embedded in the client-side source code, exposing an unprotected administrative path.

3. The hidden page was accessed by navigating to http://localhost:3000/#/score-board. The challenge was successfully completed when the Score Board page was displayed.
   Challenge completed:

This challenge demonstrates that hidden functionality in web applications can still be discovered by analysing client-side code. Even if a page is not visible in the user interface, attackers can inspect JavaScript files to find hidden routes. This highlights the importance of implementing proper access control rather than relying on obscurity.

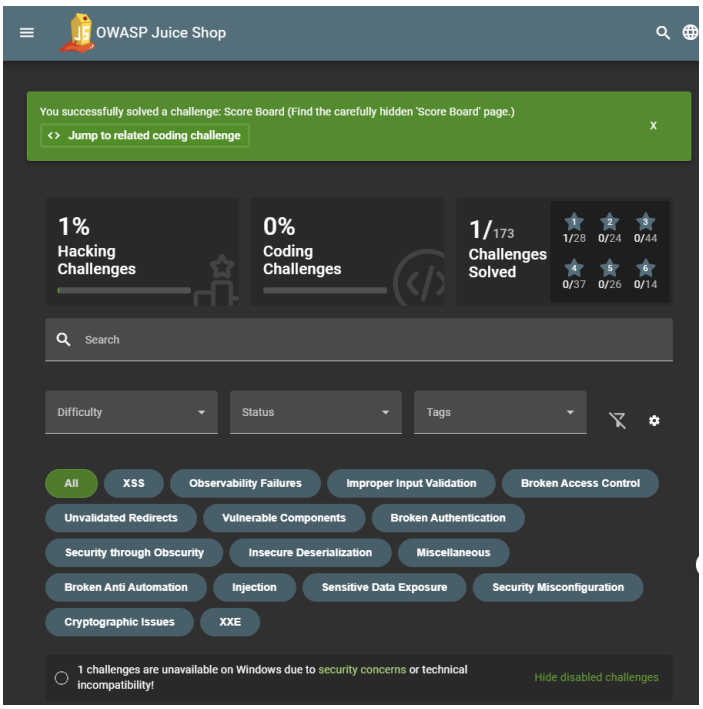

Figure 7: Score-board showing score board challenge completed

2.2 DOM XSS (Cross Site Scripting)

- Difficulty: 1 Star

Exploit a DOM-based Cross-Site Scripting vulnerability in the search feature of Juice Shop. DOM XSS means the vulnerability happens inside the browser using JavaScript, not on the server.

The search functionality of the application was identified as a potential input field where user data is processed by the browser.

A malicious JavaScript payload was inserted into the search field to test for DOM-based Cross-Site Scripting vulnerabilities. <iframe src="javascript:alert(`xss`)">

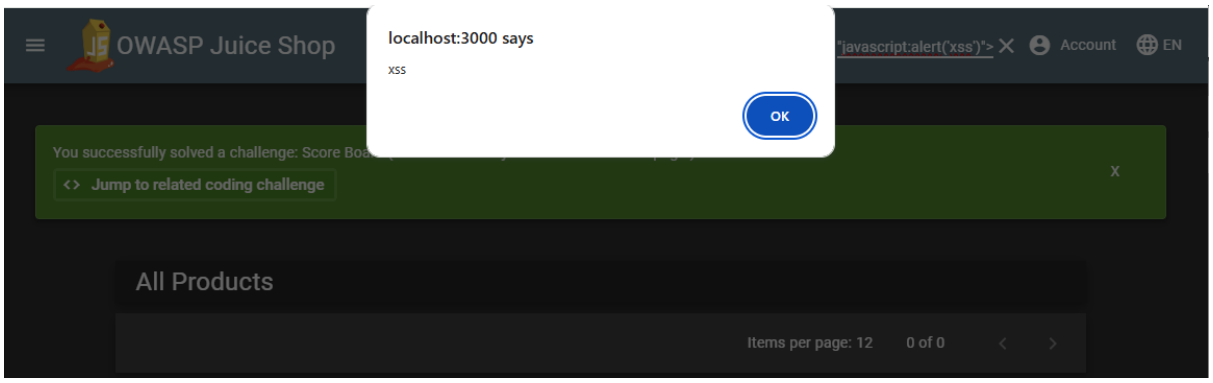

Figure 8: XSS alert triggered successfully via the search bar

The payload successfully triggered a JavaScript alert popup, confirming that the injected script was executed in the browser. Challenge completed:

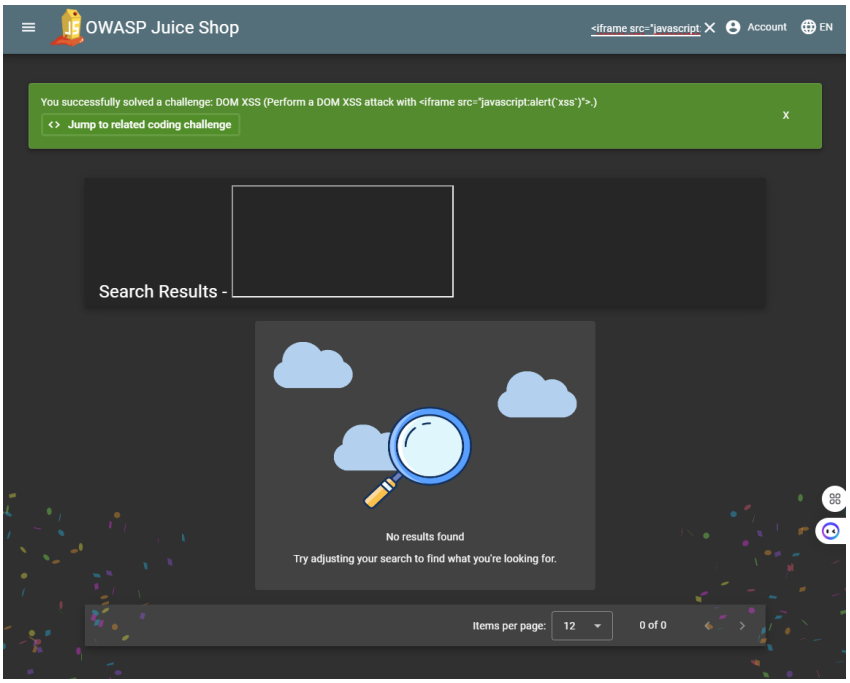

Figure 9: DOM XSS challenge successfully completed using an iframe injection payload

To prevent DOM-based XSS vulnerabilities, developers should validate and sanitise all user input before inserting it into the DOM. Using safe JavaScript methods such as textContent instead of innerHTML, implementing proper output encoding, and using Content Security Policy (CSP) can significantly reduce the risk.

2.3 Reflected XSS (Cross Site Scripting)

- Difficulty: 2 stars

Exploit a reflected cross-site scripting vulnerability in the Juice Shop search feature via the URL. Reflected XSS means:

- the malicious input is sent in the request
- the server reflects it back in the response
- the browser executes the injected script.

1. The search functionality was used to identify how user input is processed within the application.

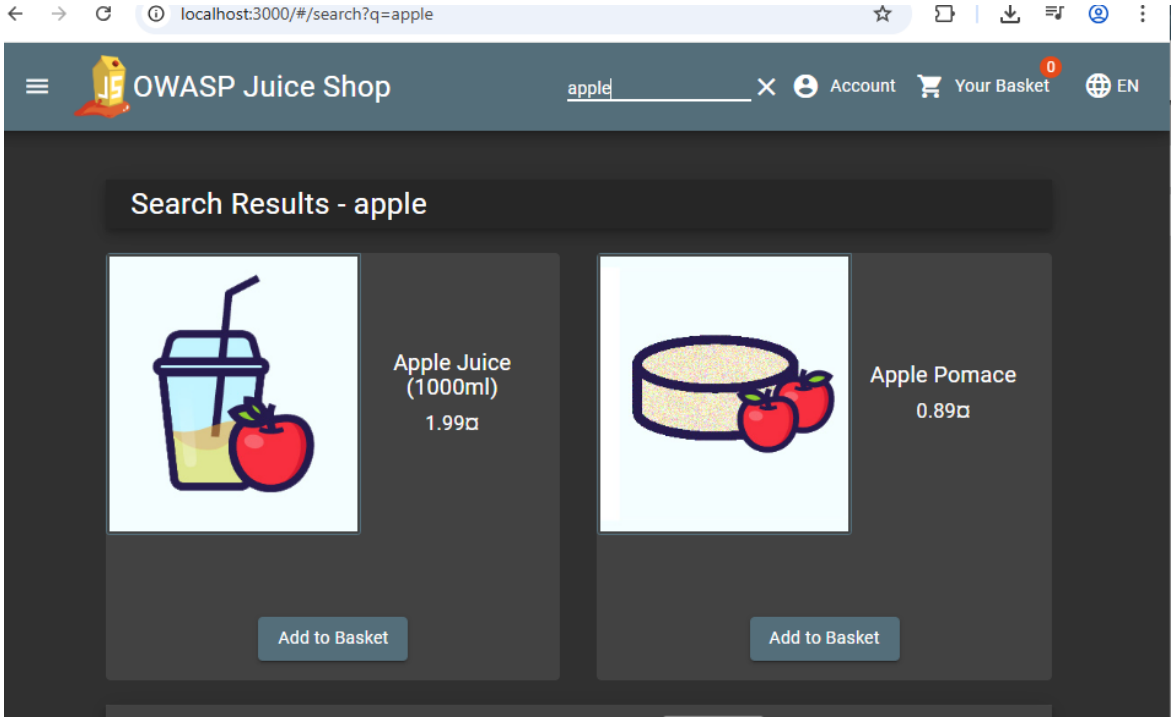

Figure 10: Search functionality returning results for "apple", showing the query reflected in the URL parameter.

2. The search parameter in the URL was modified to include a malicious JavaScript payload in order to test for reflected XSS vulnerabilities. The injected script executed successfully and triggered a JavaScript alert, confirming the presence of a reflected XSS vulnerability.

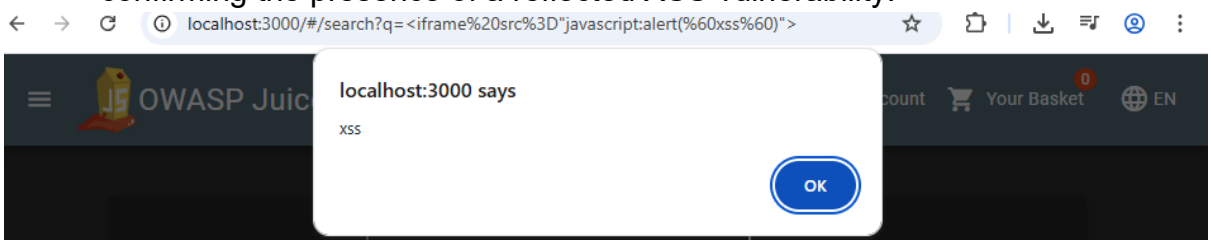

Figure 11: XSS payload injected via URL parameter, triggering an alert and confirming the search input is unsanitised

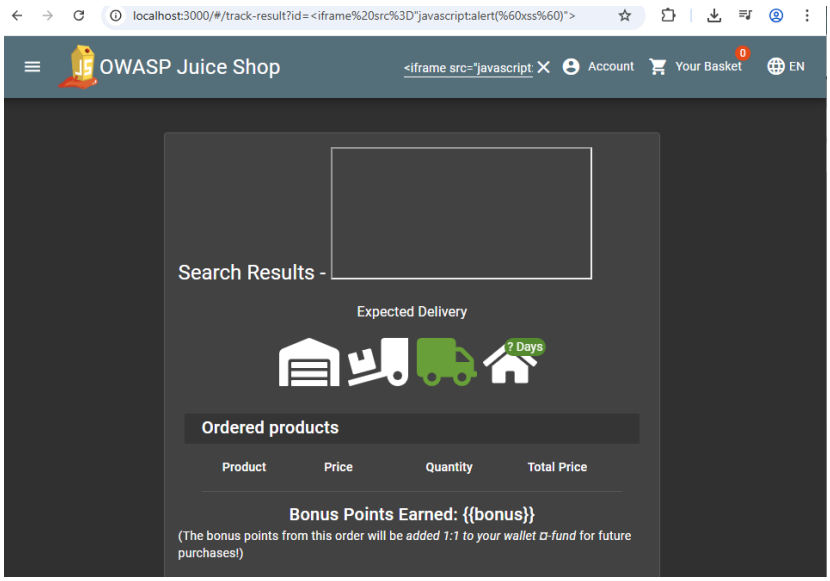

Figure 12: DOM XSS payload executing via the track-result URL

3. The challenge was successfully completed after the payload executed in the browser.

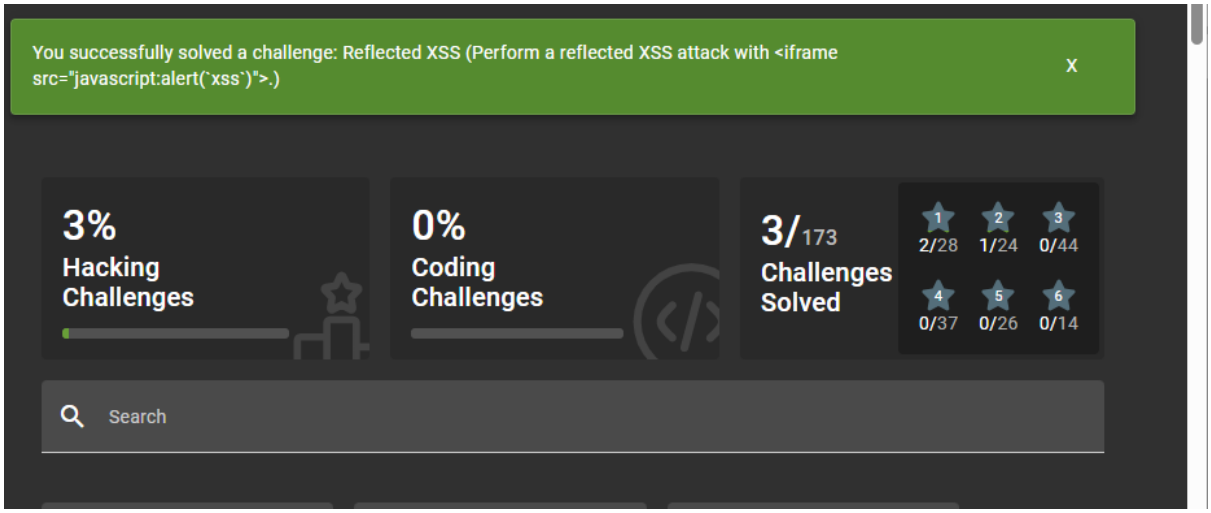

Figure 13: Reflected XSS challenge successfully completed, scoreboard showing 3 challenges solved.

Reflected Cross-Site Scripting occurs when user input from a request is immediately reflected in the server response without proper sanitisation. When the browser renders the page, the injected JavaScript executes. This allows attackers to run malicious scripts in the victim's browser.

2.4 Login MC SafeSearch

- Difficulty: 2 Stars
- Log in as the user MC SafeSearch by discovering the password that is exposed in a network request.

1. Search for MC SafeSearch

I searched for MC SafeSearch on Google to gather publicly available information about the user.

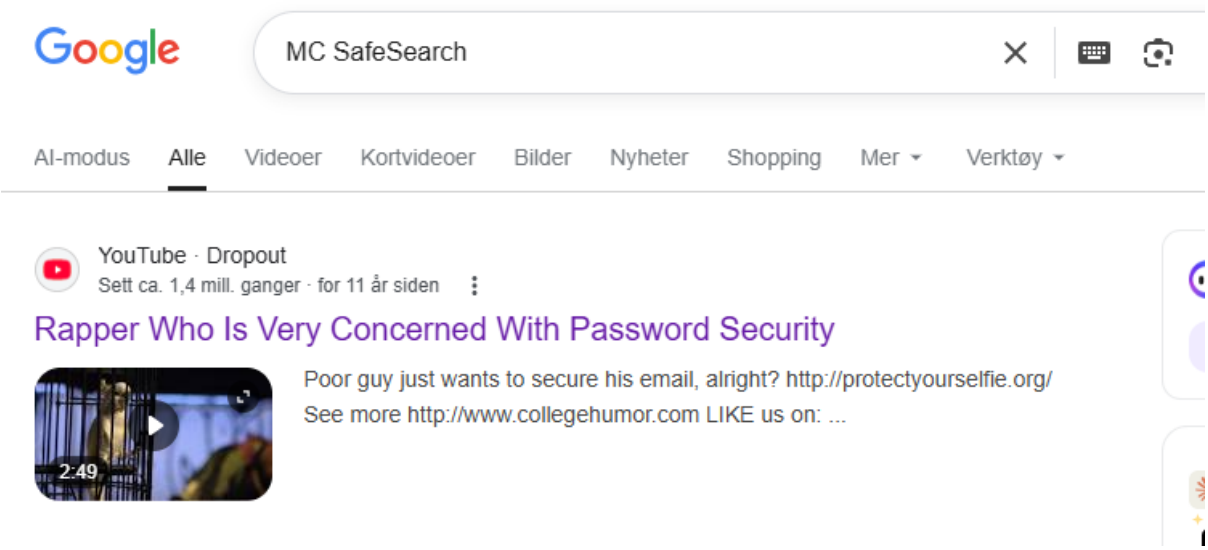

Figure 14: Google search for "MC SafeSearch" used to gather publicly available OSINT on the target user.

2. Find Password Hint in Video

In the video, a password hint is revealed mentioning the dog name “Mr. Noodles”, where the letter “o” is replaced with “0”.

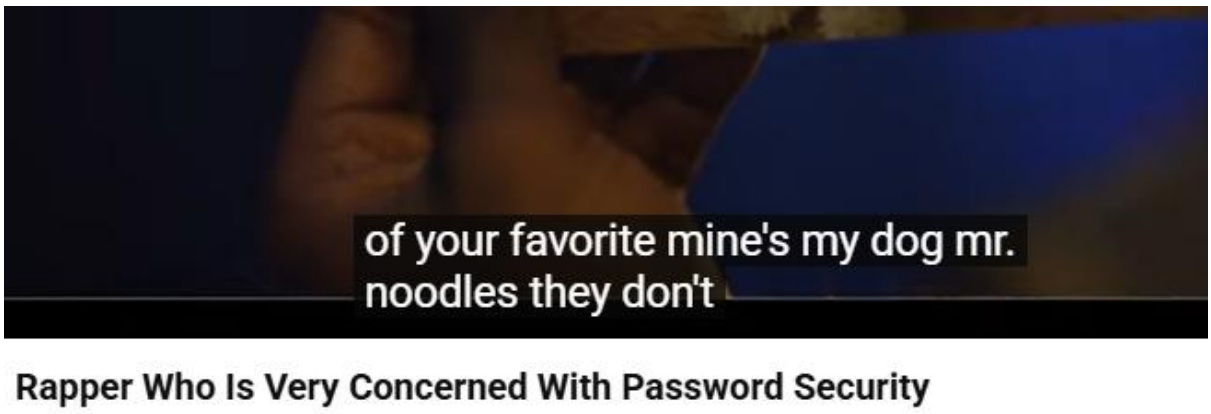

Figure 15: Video lyrics revealing the dog's name "Mr. Noodles" — used to derive the password "Mr. N00dles"

3. Log in Using Discovered Credentials

Using the username and the discovered password, I logged into OWASP Juice Shop.

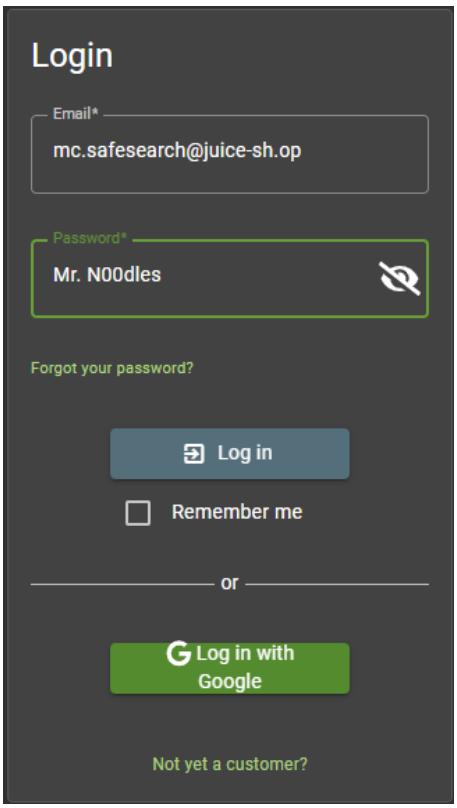

Figure 16: Logging in as MC SafeSearch using the discovered password "Mr. N00dles".

Challenge completed:

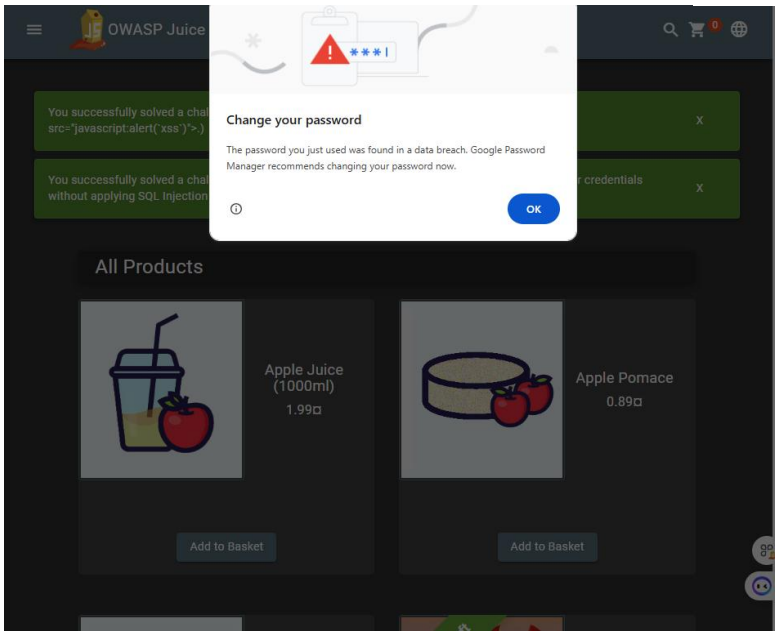

Figure 17: Login successful — browser warning confirms the password was found in a known data breach

2.5 Privacy Policy (Miscellaneous)

- Difficulty: 1 star

1. I navigated to the account-Privacy & Security section in the layout and located the Privacy Policy link. Accessing this page revealed the privacy policy information and triggered the challenge completion.

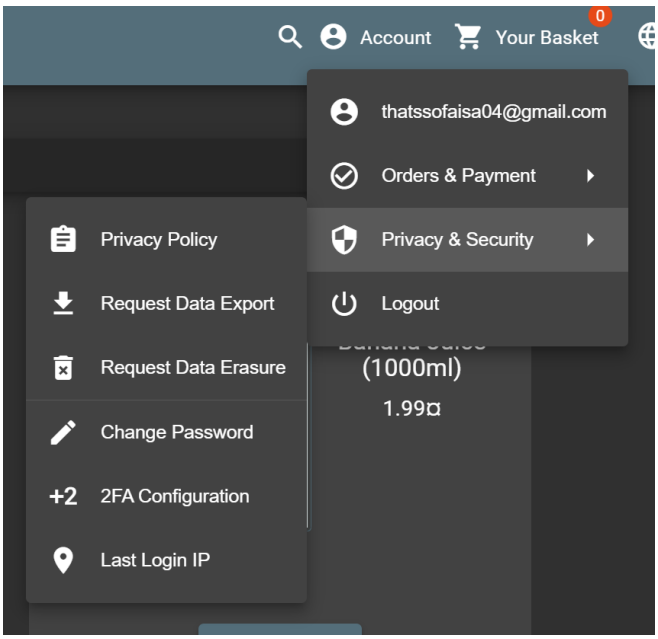

Figure 18: Account menu showing Privacy & Security section containing the Privacy Policy link.

2. After accessing the Privacy Policy page, the application automatically marked the Privacy Policy challenge as solved on the Score Board

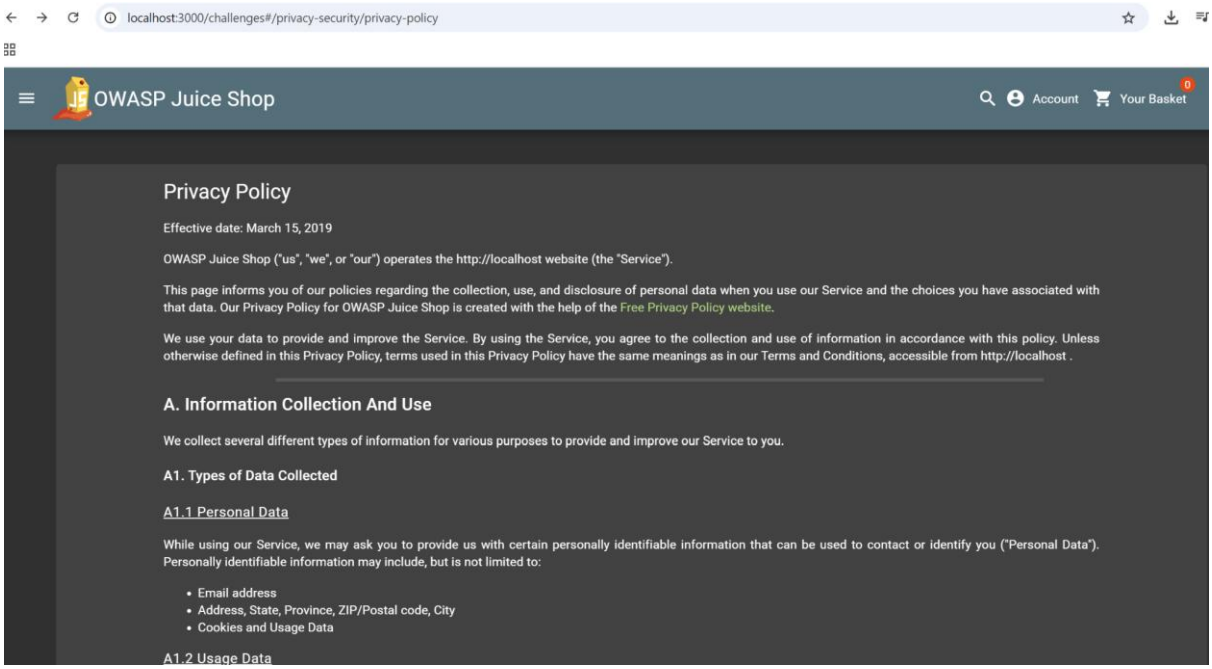

Figure 19: Privacy Policy page accessed at the privacy-security route, triggering the challenge completion

Figure 20: Successfully solced Privacy Policy challenge

The Privacy Policy challenge demonstrated the importance of reviewing publicly accessible pages in web applications. By simply navigating and accessing the Privacy Policy link, the hidden challenge was revealed. This exercise highlights that sensitive or important information can often be discovered through basic exploration of a website's structure and publicly available resources.

2.6 Login Admin (SQL Injection)

- Difficulty: 2

1. I navigated to the login page of the OWASP Juice Shop application to test for potential SQL Injection vulnerabilities in the authentication mechanism.

2. I entered the SQL injection payload `' OR 1=1--` in the email field to bypass authentication. The payload manipulated the SQL query, causing the condition to always evaluate as true and granting access to the administrator account.

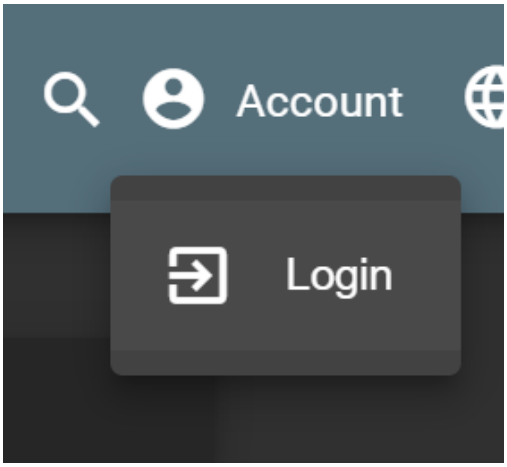

Figure 21: Login page of the OWASP Juice Shop

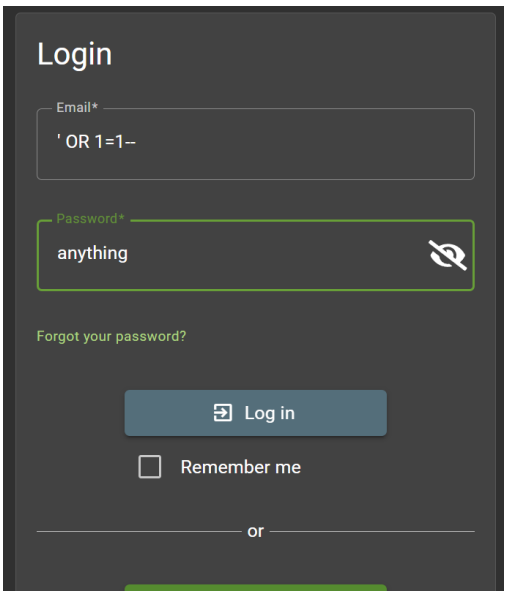

Figure 24: SQL injection payload ' OR 1=1-- entered in the email field to bypass authentication.

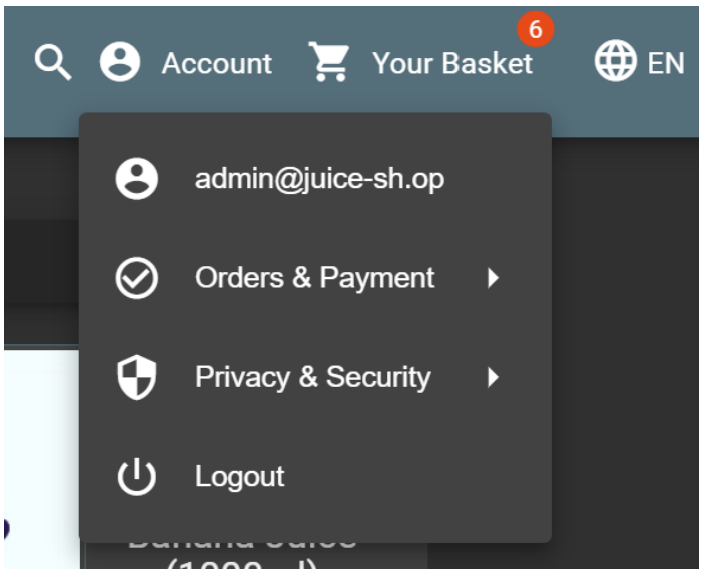

Figure 22: Admin account access granted confirming successful SQL injection login bypass.

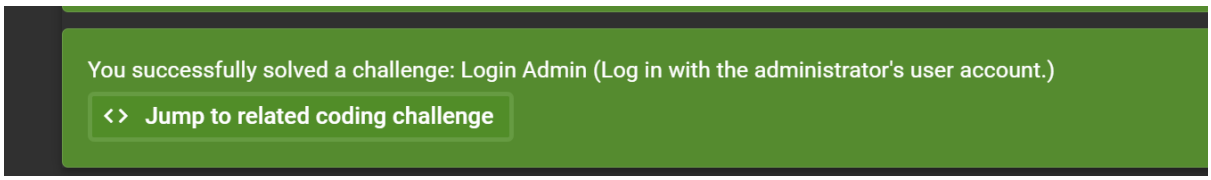

Figure 23: Successfully solved Login Admin challenge

The Login Admin challenge demonstrated a SQL Injection vulnerability in the login authentication system. By injecting the payload `' OR 1=1--` into the email field, the SQL query was manipulated to always return true, allowing authentication to be bypassed. This vulnerability occurs when user input is not properly sanitized before being included in SQL queries. Implementing parameterized queries and input validation can prevent SQL injection attacks.

2.7 Admin Section (Security Misconfiguration)

- Difficulty: 2

1. I accessed the administration panel directly by navigating to the `/administration` route in the application. The page was accessible without proper authorization, revealing a security misconfiguration.

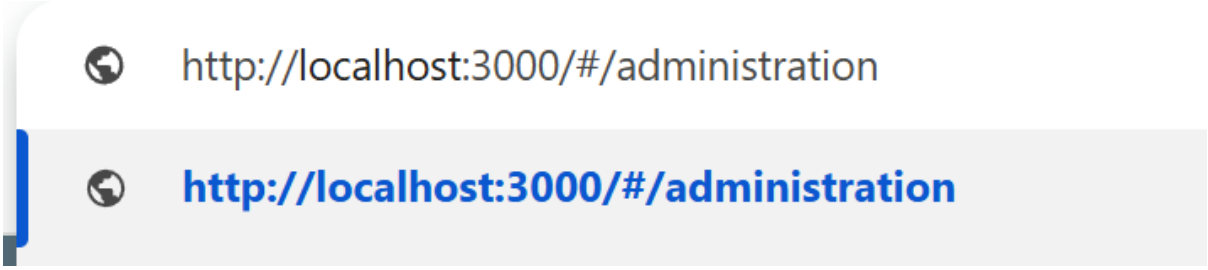

Figure 25: Administration panel accessed directly via /administration route without authorisation.

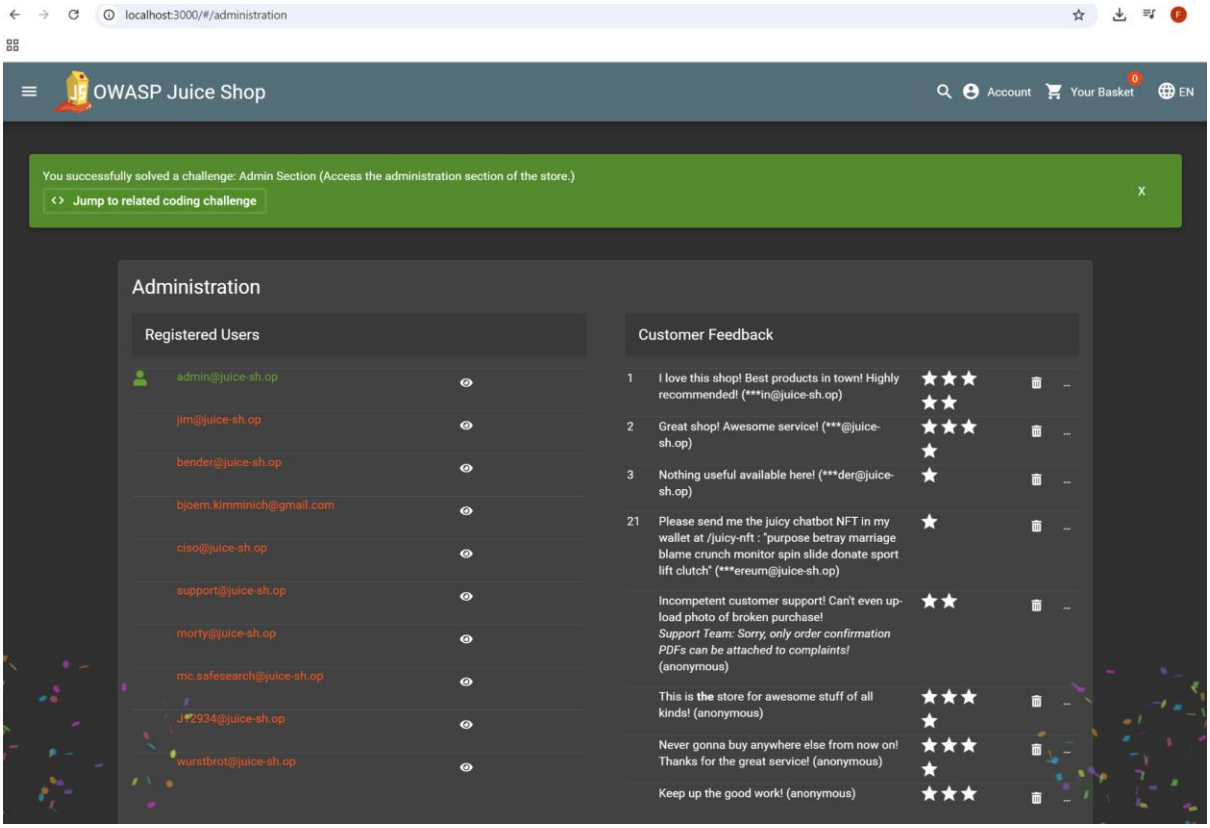

Figure 26: Admin Section challenge completed, exposing all registered users and customer feedback.

The Admin Section challenge demonstrated a security misconfiguration where the administration panel was accessible without proper authorization controls. By directly navigating to the `/administration` route, the admin interface was exposed to unauthorized users. This highlights the importance of implementing strict access control mechanisms to protect sensitive administrative functionality.

2.8 Login Jim (SQL Injection)

- Difficulty: 3
- The objective of this task is to demonstrate how a SQL Injection attack can be used to bypass the login authentication mechanism and gain access to a specific user account (Jim) without knowing the password.

1. I injected the payload `jim@juice-sh.op'--` into the email field. The single quotation mark terminates the original SQL query string, while the comment operator (`--`) causes the remaining part of the query, including the password verification, to be ignored.

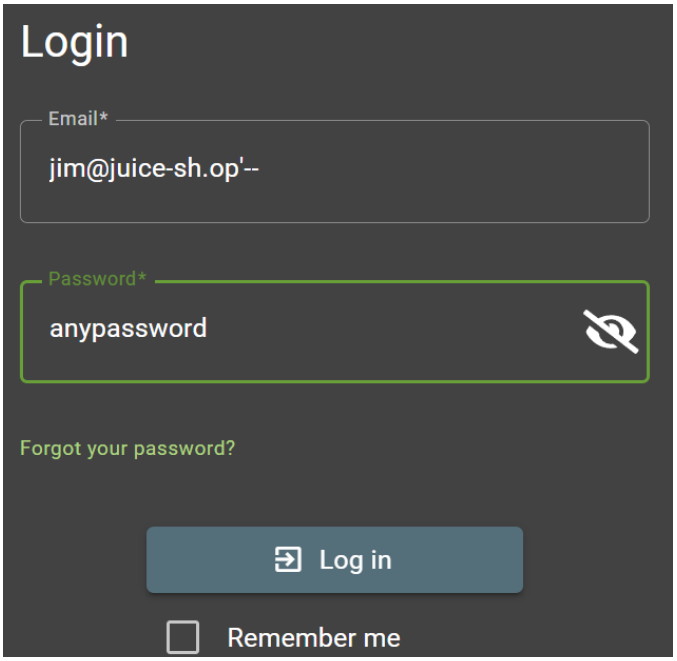

Figure 27: SQL injection payload entered targeting Jim's account using his known email to bypass authentication.

2. After submitting the malicious payload, the application authenticated the session as the user Jim without requiring the correct password. This confirmed the presence of a SQL Injection vulnerability in the login mechanism.

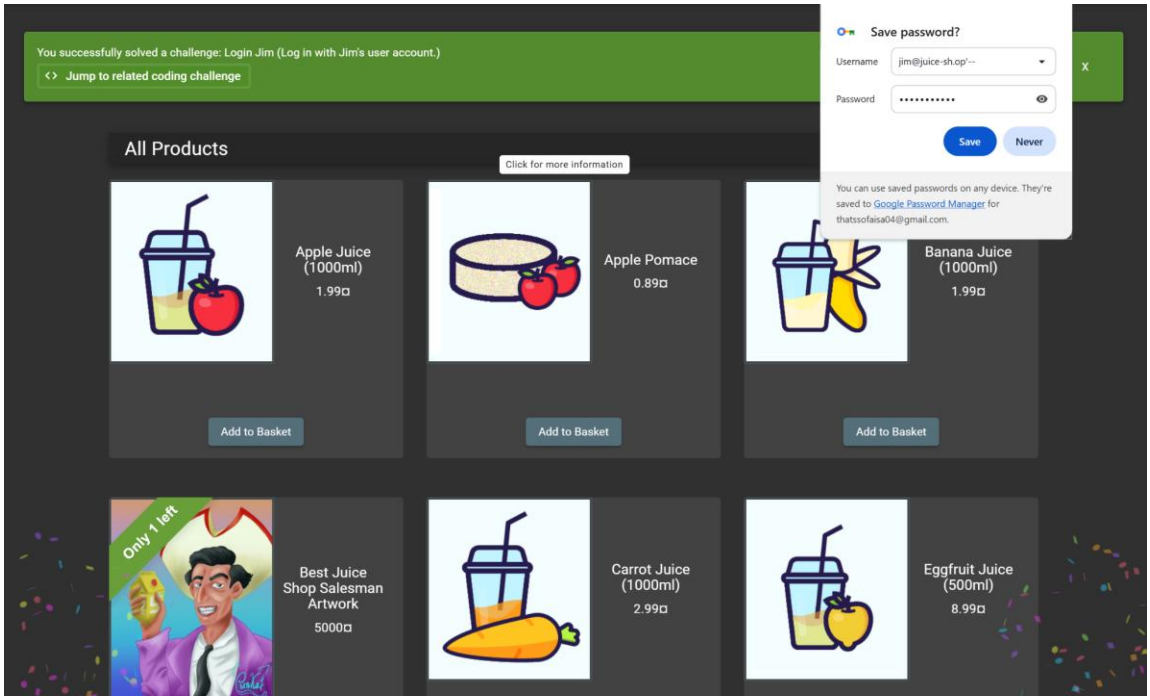

Figure 28: Login Jim challenge successfully completed via SQL injection.

This exercise demonstrated how SQL Injection can compromise authentication systems when user inputs are not properly validated. By injecting the payload `jim@juice-sh.op'--` into the login form, it was possible to bypass password verification and gain access to Jim's account. This highlights the importance of secure coding practices, including input validation and parameterized queries, to protect web applications from SQL Injection attacks.

2.9 Bjoern's Favorite Pet (Broken Authentication)

- Difficulty: 3
- The objective of this challenge is to demonstrate how weak password recovery mechanisms, such as easily guessable security questions, can allow attackers to reset a user's password and gain unauthorized access. The goal is to reset the password of the user Bjoern by answering his security question.

1. I initiated the password reset process for the user Bjoern by entering the registered email address in the password recovery form.
2. After submitting the email address, the application revealed the security question associated with Bjoern's account.

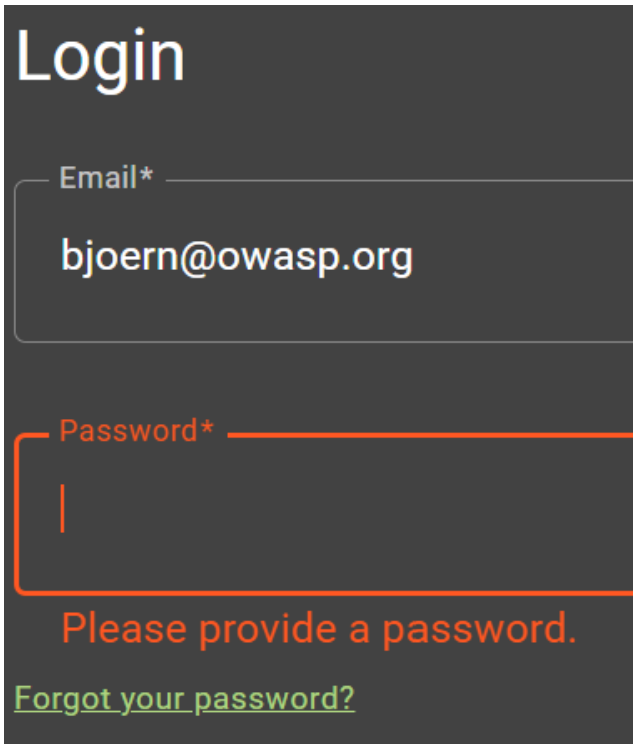

Figure 29: Login attempt for bjoern@owasp.org confirming the account exists

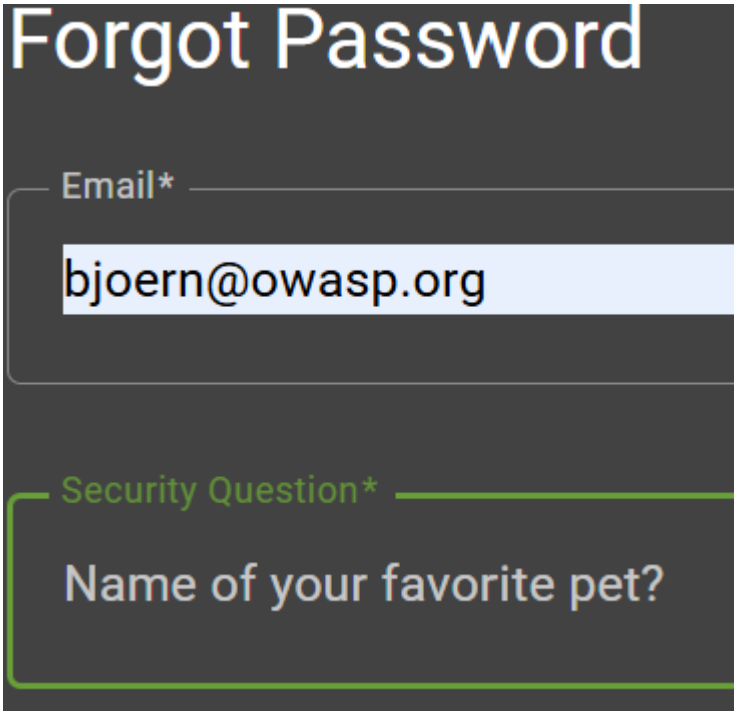

Figure 30: Forgot Password page revealing the security question "Name of your favorite pet?" for bjoern@owasp.org.

3. I Browsed the Photo Wall on Juice Shop to look for images posted by Bjoern. I found a user that looked suspicious, so I visited the twitter account.

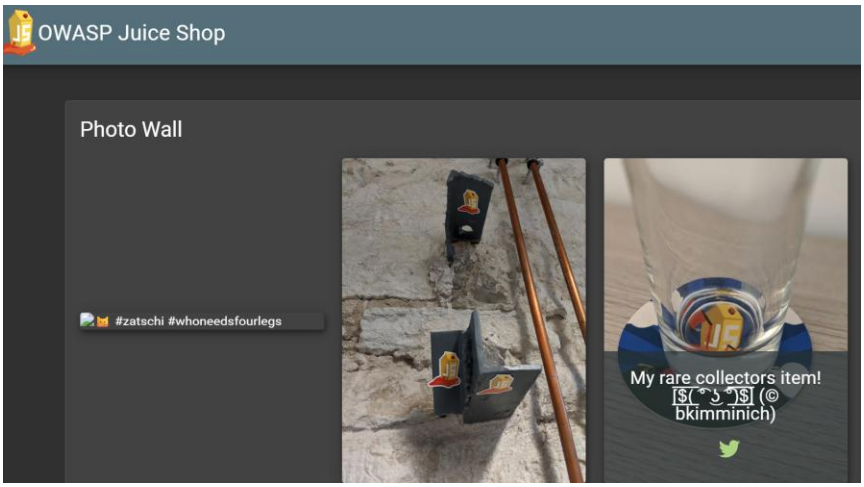

Figure 31: Photo Wall revealing the Twitter handle @bkimminich linked to Bjoern's account.

4. It was confirmed that the account belonged to Bjoern by the name. I then examined publicly visible content for hints about his pet.

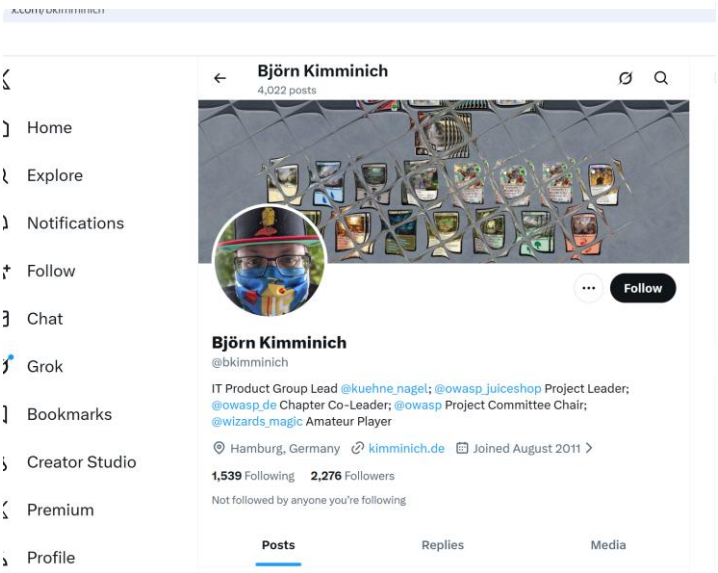

Figure 32: Björn Kimminich's public Twitter profile confirmed via OSINT.

5. I scrolled until I found a post mentioning/showing his pet. It was confirmed that the name of the pet was “Zaya”.

Figure 33: Tweet revealing Bjoern's pet name "Zaya"

6. Returned back to the password reset page and entered “Zaya” as the answer to the favorite pet question. I then created a new password for Bjoern’s account.

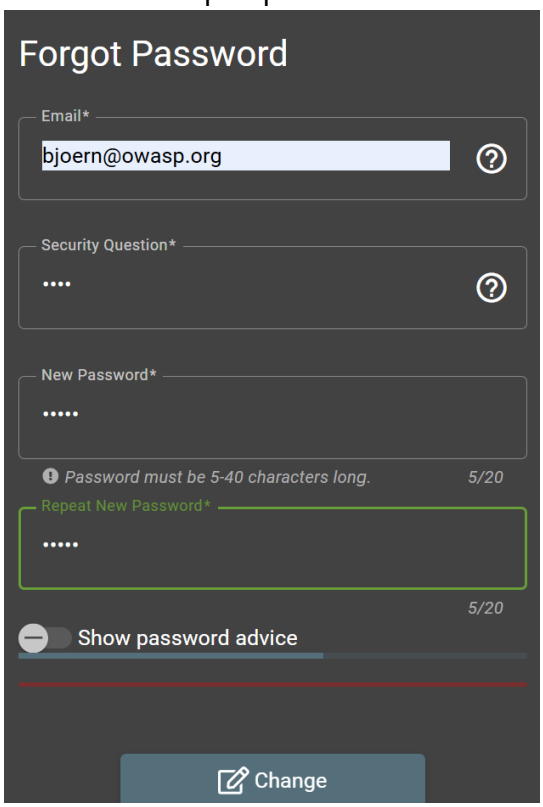

Figure 34: Password reset form completed using "Zaya" as the security question answer.

7. I logged in to the application using Bjoern's email and the newly set password, and I successfully access the account.

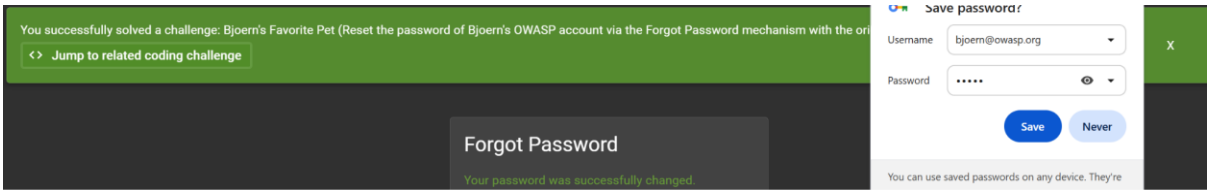

Figure 35: Successfully solved Bjoern's favorite pet challenge

The Bjoern's Favorite Pet challenge demonstrates a broken authentication vulnerability caused by weak security questions. By performing open-source intelligence (OSINT) on publicly available content, the favorite pet's name was discovered and used to reset the account password. This highlights the importance of secure password reset mechanisms and avoiding publicly guessable security questions to protect user accounts.

2.10 Kill Chatbot (Vulnerable Components)

- Difficulty: 5
- The objective of this challenge is to demonstrate how a **vulnerable third-party component**, such as a chatbot library, can be manipulated to disable a feature of the application.
- The goal is to **disable the chatbot functionality** within the application.

1. Investigating the Chatbot Code

To understand how the chatbot works, I visited the Juice Shop GitHub repository and opened the juicy-chat-bot package. Inside this package, I opened the file: index.ts. While reviewing the code, I noticed the following line: import vm from 'vm2'

The vm2 library allows code to run inside a virtual machine (sandbox). This means the chatbot can execute user input inside this virtual environment.

However, if user input is not properly sanitized, attackers might be able to inject code into the virtual machine, which can manipulate how the chatbot behaves.

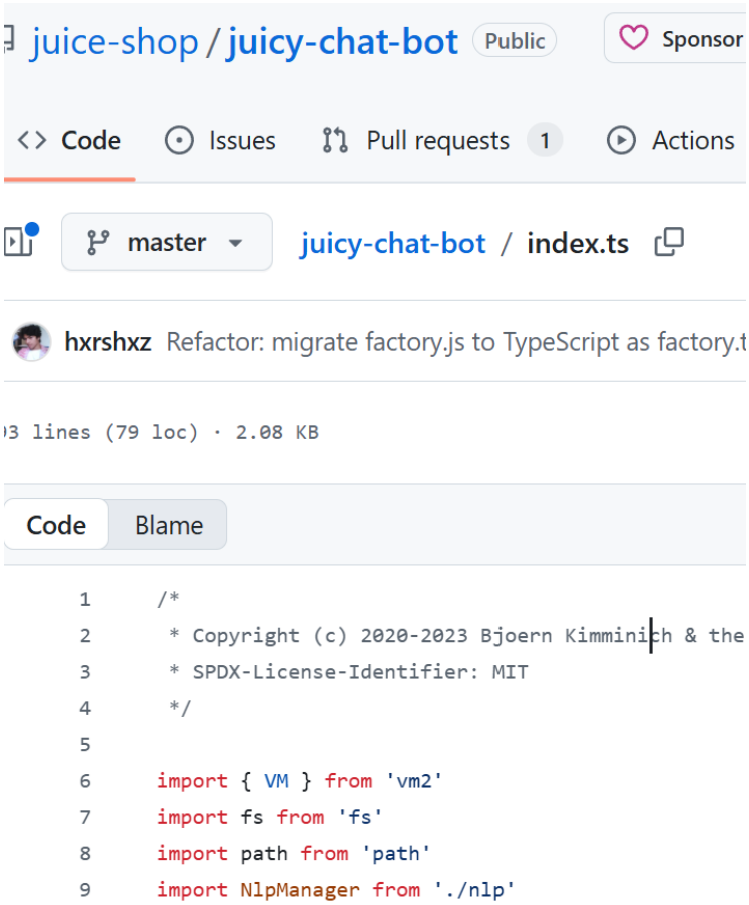

Figure 36: Juice Shop's public GitHub repository revealing the chatbot source code for analysis.

2. Finding Where the Virtual Machine Is Used

Next, I searched the file for **"vm"** to see how it was used. I found that the application creates a virtual machine instance: this.factory = new vm(...)

```
49        this.factory = new VM({
50          sandbox: {
51            Nlp: NlpManager,
52            training: this.training
53          }
54        })
55        this.factory.run(ctx)
56        this.factory.run(`trainingSet=${trainingSet}`)
57      }
```

Figure 37: Search for "vm" revealing the vulnerable VM instance in the chatbot code.

3. Reviewing the addUser Method

After locating the virtual machine declaration, I scrolled further down the file and found the addUser method. Inside this method, the chatbot passes the username message into the virtual machine process.

```
68      addUser (token: string, name: string): void {
69        this.factory.run(`users.addUser("${token}", "${name}")`)
70      }
```

Figure 38: addUser method passing unsanitised user input directly into the VM

This behavior is similar to using an eval statement, meaning the application executes user input as code. If an attacker sends a specially crafted message, they may be able to modify the chatbot's internal logic.

4. Exploiting the Vulnerability

Next, I opened the Support Chat feature inside the Juice Shop application. When the chatbot asked for my name, I entered: /kill

Then I entered the following payload:

admin"); process = null; users.addUser("1337", /kill

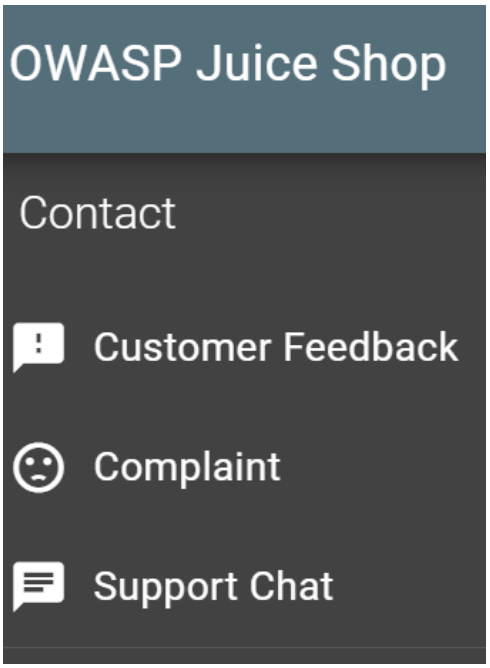

Figure 39: Navigation menu showing the Support Chat option

Because the chatbot executes user input inside the vm2 virtual machine, the payload injects code into the chatbot's internal process. By setting `process` to `null`, the chatbot can no longer process messages, which effectively kills the chatbot.

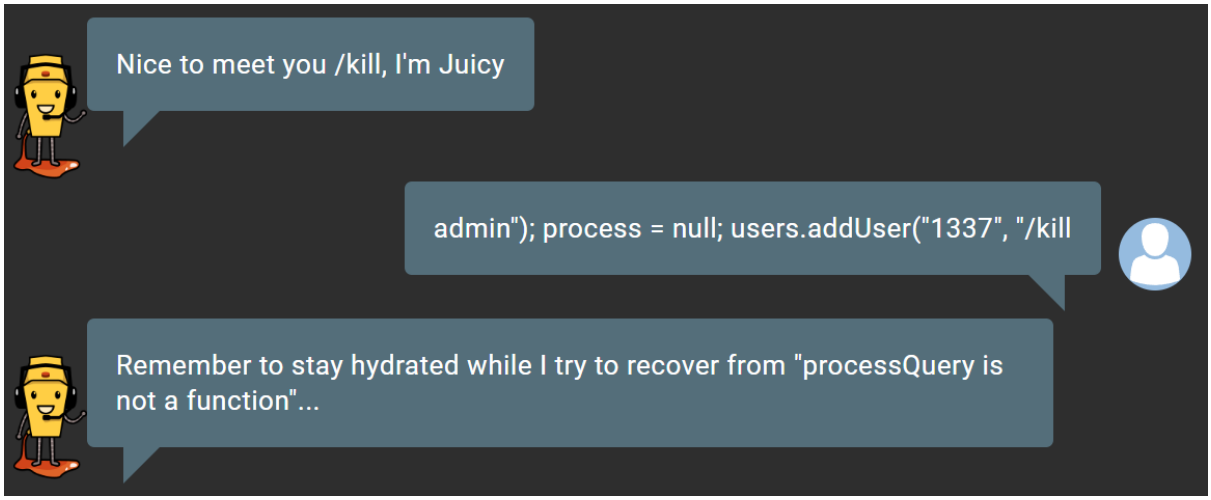

Figure 40:VM injection payload sent via chatbot, crashing the process and confirming the vulnerability.

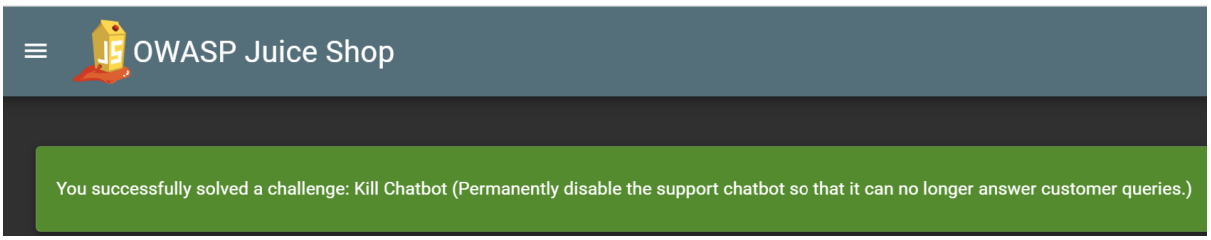

Figure 41: Successfully solved kill chatbot challenge

Conclusion

This challenge demonstrated how vulnerable third-party components can introduce security risks into an application. The chatbot used the vm2 virtual machine to process user input but did not properly validate or sanitize the input before executing it.

By injecting a specially crafted payload, it was possible to alter the chatbot's internal process and disable its functionality. This shows the dangers of executing user input directly and highlights the importance of secure input validation and careful use of external libraries.

Proper security measures such as input sanitization, strict sandboxing, and avoiding unsafe execution of user input are necessary to prevent these types of vulnerabilities

Reflection

Working through the Juice Shop challenges gave me practical exposure to a range of attack types including XSS, SQL injection, OSINT-based account takeover, and VM injections each requiring a different mindset and approach.

What stood out most was how small misconfigurations lead to significant consequences. The SQL injection bypass required only a single line of input yet granted full admin access. Similarly, the MC Safe Search and Bjoern challenges showed how much sensitive information is publicly available through social media, reinforcing how important OSINT awareness is from a security perspective.

Overall this lab significantly improved my practical understanding of web application security.

Use of Artificial Intelligence

In accordance with the University of Roehampton's AI guidelines, the following describes how AI tools were used to support the completion of this assessment.

ChatGPT was used for the following tasks:

- Reviewing and improving the overall organization and flow of the report to ensure it meets the assessment brief requirements.
- Debugging assistance during the lab environment setup, providing guidance on configuring the virtual machine network adapters and troubleshooting connectivity issues between the Kali Linux and WordPress VMs.

All technical work, screenshots, findings, and analysis represent my own independent lab work. AI was not used to generate technical findings, perform scans, or interpret screenshots.